THAPAR INSTITUTE OF ENGINEERING & TECHNOLOGY

(Deemed to be University), Patiala

Final Project Report

UCS310 – Database Management Systems

BrickYield

A Fractional Real Estate Investment Platform

Submitted by

1024170101	**Harsimrat Singh Malhotra**
1024170102	**Aviraj Singh**
1024170103	**Pehal**

Submitted to

Department of Computer Science & Engineering

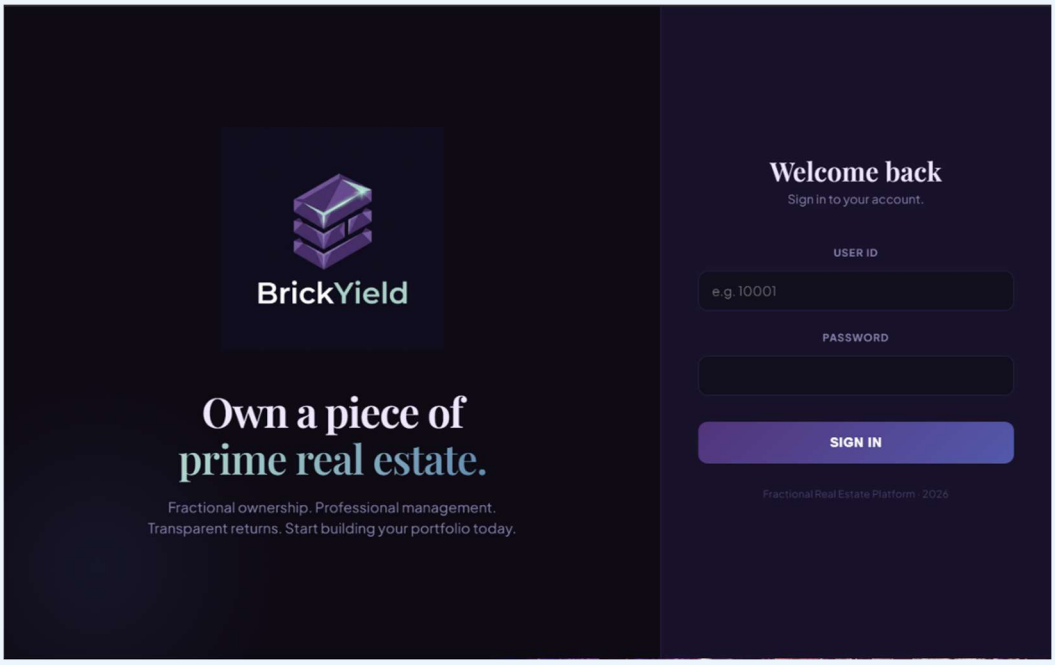
INDEX

1. Introduction

BrickYield is a full-stack fractional real estate investment platform that democratizes access to commercial and residential property assets. Traditional real estate investment demands significant capital—often several lakhs to crores of rupees—placing it out of reach for the average investor. BrickYield solves this by dividing each property into thousands of digital shares that can be purchased individually, enabling investors to diversify across multiple properties with minimal capital.

The platform directly connects two classes of users: verified real estate Developers who list properties and manage operations, and Investors who browse the marketplace, purchase shares, and earn proportional dividend income derived from rental revenue. A built-in wallet system, KYC verification workflow, and transparent dividend-distribution pipeline ensure compliance and trust.

The backend is engineered on MySQL 8.0 (InnoDB engine) with a rich set of stored procedures, views, and business-logic constraints. A FastAPI layer exposes RESTful endpoints consumed by a ReactJS single-page application, resulting in a complete three-tier architecture.



This report documents the complete database design, normalization analysis, PL/SQL implementation, and system architecture underpinning BrickYield, submitted as the final project for UCS310 – Database Management Systems.

2. Problem Statement

Real estate has historically been the highest-returning asset class in India. However, several structural barriers prevent ordinary investors from participating:

- High capital barrier — a single residential or commercial property may cost ₹50 lakh to several crores, excluding the majority of retail investors.
- Illiquidity — once invested, capital is locked until the entire property is sold.
- Lack of transparency — rental income, maintenance costs, and occupancy rates are rarely disclosed to passive investors.
- Concentration risk — investors who can afford real estate typically own one or two properties, making diversification impractical.
- Operational overhead — property management, tenant dealings, and regulatory compliance are burdensome for small investors.

There is a clear need for a technology-driven platform that

(a) lowers the minimum investment threshold to a few hundred rupees per share,

(b) provides complete financial transparency through structured data,

(c) automates dividend computation and distribution, and

(d) maintains a secure, auditable transaction record.

BrickYield addresses all four requirements through a relational database-driven backend.

3. Objectives

The specific objectives of the BrickYield project are:

1. Design a normalized relational database schema (up to 3NF) covering all entities: Users, Investors, Developers, Properties, Tenants, Lease Agreements, Share Ownership, Share Transactions, Dividend Distributions, Dividend Payments, Maintenance Expenses, and Revenue Records.
2. Model entity relationships through an ER Diagram and derive a corresponding relational schema with clearly defined primary keys, foreign keys, and domain constraints.
3. Implement DDL commands (CREATE TABLE, ALTER TABLE, CREATE INDEX) for the complete schema.
4. Populate the database with representative seed data using DML (INSERT, UPDATE) statements.
5. Write advanced SELECT queries using multi-table JOINs, subqueries, GROUP BY / HAVING, and window functions.
6. Build SQL Views (investor_portfolio_view, property_marketplace_view, developer_property_detail_view) to simplify complex queries for the API layer.
7. Develop stored procedures—purchase_shares, get_investor_profile, get_developer_profile, process_dividend_payout, get_admin_system_report—to encapsulate core business logic.
8. Implement transaction management using START TRANSACTION, COMMIT, and ROLLBACK with proper SQLEXCEPTION handlers.
9. Expose the database through a FastAPI backend and consume it in a ReactJS frontend, demonstrating a complete three-tier architecture.
10. Optimize performance through strategic indexing on all foreign-key columns.

4. Scope of the Project

4.1 Functional Scope

The system supports the following functional capabilities:

- Investor registration, KYC verification tracking, and wallet management.
- Developer onboarding with company registration and property listing.
- Property marketplace with share price, availability, and developer details.
- Share purchase transactions with real-time wallet deduction and ownership recording.
- Dividend distribution pipeline — revenue aggregation, expense deduction, per-share dividend calculation, and individual investor payment.
- Admin system report with platform-wide statistics.
- Financial history for both investors (transactions + dividend payments) and developers (revenue records).

4.2 User Roles

- Admin — platform oversight, KYC approval, system reporting.
- Developer — property listing, lease management, revenue tracking.
- Investor — browse marketplace, purchase shares, view portfolio and dividends.

4.3 Limitations

- Secondary market (peer-to-peer share trading) is designed in the schema but partially implemented.
- No real payment gateway integration — wallet top-up is simulated.
- Email/SMS notification service is out of scope.
- Property valuation and automated re-pricing are not included in the current version.

5. System Overview & Architecture

5.1 System Architecture

BrickYield follows a three-tier architecture separating presentation, business logic, and data layers.

s

```
brickyield/
├── brickyield-frontend/     # React source code & Vite config
│   ├── src/
│   │   ├── App.jsx          # Main application logic & UI
│   │   └── ...
│   └── package.json
├── brickyield-backend/      # FastAPI server & logic
│   ├── main.py              # Entry point
│   ├── database.py          # MySQL connection setup
│   ├── routers/             # API endpoints (investor, developer, admin)
│   └── requirements.txt
└── database/
    └── schema.sql           # Database export including tables & procedures
```

5.2 Technology Stack

Layer	Technology	Purpose
Frontend	ReactJS (Vite)	Single-page application — marketplace, portfolio, developer dashboard
Backend API	FastAPI (Python)	RESTful endpoints; calls MySQL stored procedures and views
Database	MySQL 8.0 (InnoDB)	Relational schema, stored procedures, views, indexing
Auth	JWT Tokens	Stateless authentication for investor, developer, admin roles
Dev Tools	VS Code, MySQL Workbench	Development, schema design, API testing

5.3 System Workflow

The end-to-end user journey on BrickYield proceeds as follows:

11. User registers with email, password, and role (Investor / Developer).
12. Admin verifies KYC — sets kyc_verified = 1 in the Users table.
13. Developer lists properties with share price, total shares, area, and city details.
14. Developer adds tenant and lease information; revenue records are updated monthly.
15. Investor browses the property_marketplace_view, selects a property, and calls purchase_shares().
16. purchase_shares() validates wallet balance and share availability, then atomically updates wallet, share ownership, and share transaction log.
17. Admin triggers process_dividend_payout(), which iterates over all distributions, inserts dividend_payments records, and credits investor wallets.
18. Investors view real-time portfolio value and dividend history through get_investor_profile() and get_investor_financial_history().

6. Database Design

.

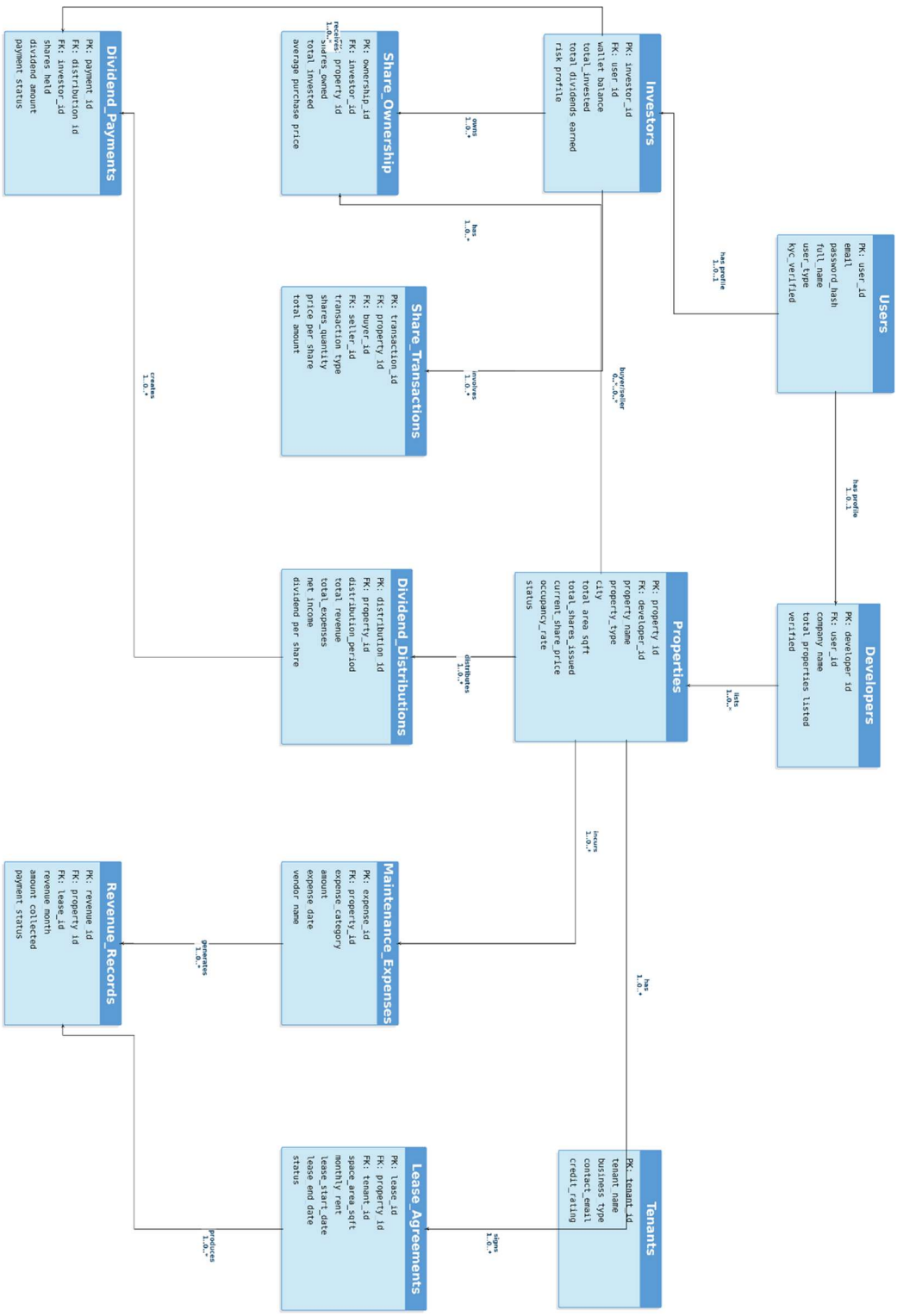
6.1 Entities and Attributes

The following table summarises the main entities:

Entity	Primary Key	Key Attributes
Users	user_id	full_name, email, password_hash, user_type, kyc_verified, gender
Investors	investor_id	user_id (FK), wallet_balance, total_invested, total_dividends_earned, risk_profile, bank_account_number
Developers	developer_id	user_id (FK), company_name, company_registration_number, total_properties_listed, verified
Properties	property_id	developer_id (FK), property_name, property_type, city, state, total_area_sqft, total_shares_issued, available_shares, current_share_price
Tenants	tenant_id	tenant_name, business_type, contact_email, credit_rating
Lease_Agreements	lease_id	property_id (FK), tenant_id (FK), space_area_sqft, monthly_rent, lease_start_date, lease_end_date
Maintenance_Expenses	expense_id	property_id (FK), amount
Dividend_Distributions	distribution_id	property_id (FK), total_revenue, total_expenses, net_income, dividend_per_share
Share_Ownership	ownership_id	investor_id (FK), property_id (FK), shares_owned, total_invested, average_purchase_price
Share_Transactions	transaction_id	property_id (FK), buyer_id (FK), seller_id (FK), transaction_type, shares_quantity, price_per_share, total_amount, transaction_date
Dividend_Payments	payment_id	distribution_id (FK), investor_id (FK), shares_held, dividend_amount, payment_status
Revenue_Records	revenue_id	property_id (FK), lease_id (FK), revenue_month, amount_collected, payment_status

6.2 Entity–Relationship (ER) Diagram

The ER model defines 12 entities interconnected through well-defined relationships. The diagram below captures all entities, their primary key attributes, and cardinality constraints



6.3 Relational Schema

The relational schema derived from the ER model is presented below, with primary and foreign key notations:

```
USERS (user_id PK, full_name, gender, email UNIQUE, password_hash, user_type,
kyc_verified)
INVESTORS (investor_id PK, user_id FK→Users, wallet_balance, total_invested,
           total_dividends_earned, risk_profile, bank_account_number)
 DEVELOPERS (developer_id PK, user_id FK→Users, company_name,
             company_registration_number UNIQUE, total_properties_listed, verified)
PROPERTIES (property_id PK, developer_id FK→Developers, property_name,
property_type,
            city, state, total_area_sqft, total_shares_issued, available_shares,
current_share_price)
TENANTS (tenant_id PK, tenant_name, business_type, contact_email, credit_rating)
LEASE_AGREEMENTS (lease_id PK, property_id FK→Properties, tenant_id FK→Tenants,
                  space_area_sqft, monthly_rent, lease_start_date, lease_end_date)
MAINTENANCE_EXPENSES (expense_id PK, property_id FK→Properties, amount)
DIVIDEND_DISTRIBUTIONS (distribution_id PK, property_id FK→Properties,
                        total_revenue, total_expenses, net_income,
dividend_per_share)
SHARE_OWNERSHIP (ownership_id PK AUTO_INCREMENT, investor_id FK→Investors,
                 property_id FK→Properties, shares_owned, total_invested,
average_purchase_price)
SHARE_TRANSACTIONS (transaction_id PK AUTO_INCREMENT, property_id FK→Properties,
                    buyer_id FK→Investors, seller_id FK→Investors,
transaction_type,
                    shares_quantity, price_per_share, total_amount,
transaction_date)
DIVIDEND_PAYMENTS (payment_id PK AUTO_INCREMENT, distribution_id
FK→Dividend_Distributions,
                   investor_id FK→Investors, shares_held, dividend_amount,
payment_status)
REVENUE_RECORDS (revenue_id PK, property_id FK→Properties, lease_id
FK→Lease_Agreements,
                 revenue_month, amount_collected, payment_status)
```

Key Relationships

- Users ←1:1→ Investors (one investor profile per investor user)
- Users ←1:1→ Developers (one developer profile per developer user)
- Developers ←1:N→ Properties (one developer lists many properties)
- Properties ←1:N→ Lease_Agreements (one property can have many leases)
- Tenants ←1:N→ Lease_Agreements (one tenant can occupy space in many properties)
- Properties ←1:N→ Maintenance_Expenses, Dividend_Distributions, Revenue_Records
- Investors ←M:N→ Properties via Share_Ownership (many investors per property, many properties per investor)
- Dividend_Distributions ←1:N→ Dividend_Payments (one distribution generates many individual payments)

7. Normalization

Normalization is a systematic process to reduce data redundancy and improve integrity by organizing data into well-structured relations. The BrickYield schema is designed up to Third Normal Form (3NF).

7.1 First Normal Form (1NF) — Atomicity

A relation is in 1NF if every attribute contains only atomic (indivisible) values and there are no repeating groups.

- Each column in every table stores exactly one atomic value (e.g., city is a single string, not a list).
- Investor holdings are not stored as a comma-separated list inside the Users table; instead each holding is a separate row in Share_Ownership.
- Multiple lease agreements for the same property are individual rows in Lease_Agreements — not multi-valued columns.
- All tables have well-defined primary keys, guaranteeing row uniqueness.

7.2 Second Normal Form (2NF) — No Partial Dependencies

A relation is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the entire primary key.

- All tables use single-column surrogate primary keys (user_id, property_id, etc.), so partial dependency cannot arise by definition.
- In Share_Ownership, the composite unique key (investor_id, property_id) governs shares_owned, total_invested, and average_purchase_price — no attribute depends on only one part of the key.
- Dividend_Payments stores shares_held and dividend_amount per (distribution_id, investor_id) — each is dependent on the full composite, satisfying 2NF.

7.3 Third Normal Form (3NF) — No Transitive Dependencies

A relation is in 3NF if it is in 2NF and no non-key attribute is transitively dependent on the primary key through another non-key attribute.

- Property details (city, area, price) are stored only in Properties and referenced by foreign key in Share_Ownership and Share_Transactions — never duplicated.
- Developer company_name is stored only in Developers; Bookings and Properties reference developer_id, preventing transitive dependency.
- Tenant details (tenant_name, business_type) reside only in Tenants; Lease_Agreements holds tenant_id as FK.
- ORDER_ITEMS style: Share_Transactions captures price_per_share at time of purchase to preserve historical accuracy — this denormalization is intentional and does not violate 3NF since it represents a historical fact, not a derived value.

7.4 Benefits of Normalization in BrickYield

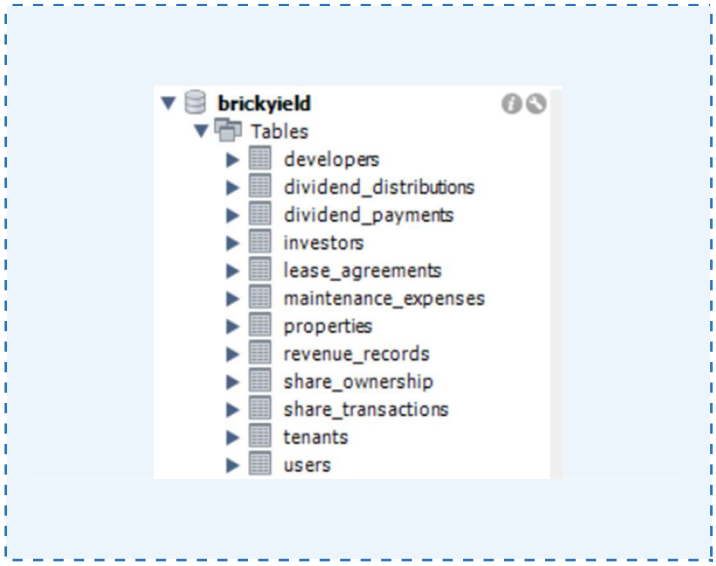
- A change to a developer's company_name requires updating only one row in Developers — all related queries via JOIN reflect the change automatically.
- Deletion of a lease record does not remove property or tenant master data.
- New investor registration cannot accidentally create orphaned share ownership records thanks to FK constraints.
- Reduced storage footprint — property details are stored once regardless of how many investors own shares in it.

8. Database Implementation — DDL

8.1 Table Creation

All tables are created using MySQL 8.0 InnoDB with UTF-8 character set. The creation order respects foreign key dependencies.

```
CREATE TABLE IF NOT EXISTS Users (
    user_id       INT           NOT NULL,
    full_name     VARCHAR(100)  NOT NULL,
    gender        ENUM('male','female')          NOT NULL,
    email         VARCHAR(150)  NOT NULL,
    password_hash VARCHAR(255)  NOT NULL,
    user_type     ENUM('investor','developer','admin') NOT NULL,
    kyc_verified  TINYINT(1)    NOT NULL DEFAULT 0,
    CONSTRAINT pk_users       PRIMARY KEY (user_id),
    CONSTRAINT uq_users_email UNIQUE      (email),
    CONSTRAINT chk_users_kyc  CHECK (kyc_verified IN (0,1))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS Investors (
    investor_id            INT           NOT NULL,
    user_id                INT           NOT NULL,
    wallet_balance         DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    total_invested         DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    total_dividends_earned DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    risk_profile           ENUM('conservative','moderate','aggressive') NOT NULL
DEFAULT 'moderate',
    bank_account_number    VARCHAR(20)   NOT NULL,
    CONSTRAINT pk_investors         PRIMARY KEY (investor_id),
    CONSTRAINT uq_investors_user    UNIQUE (user_id),
    CONSTRAINT fk_investors_user    FOREIGN KEY (user_id) REFERENCES Users(user_id)
                                    ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_investors_wallet CHECK (wallet_balance >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```



(Similar CREATE TABLE statements exist for Developers, Properties, Tenants, Lease_Agreements, Maintenance_Expenses, Dividend_Distributions, Share_Ownership, Share_Transactions, Dividend_Payments, and Revenue_Records — see schema.sql for complete DDL.)

8.2 Indexes

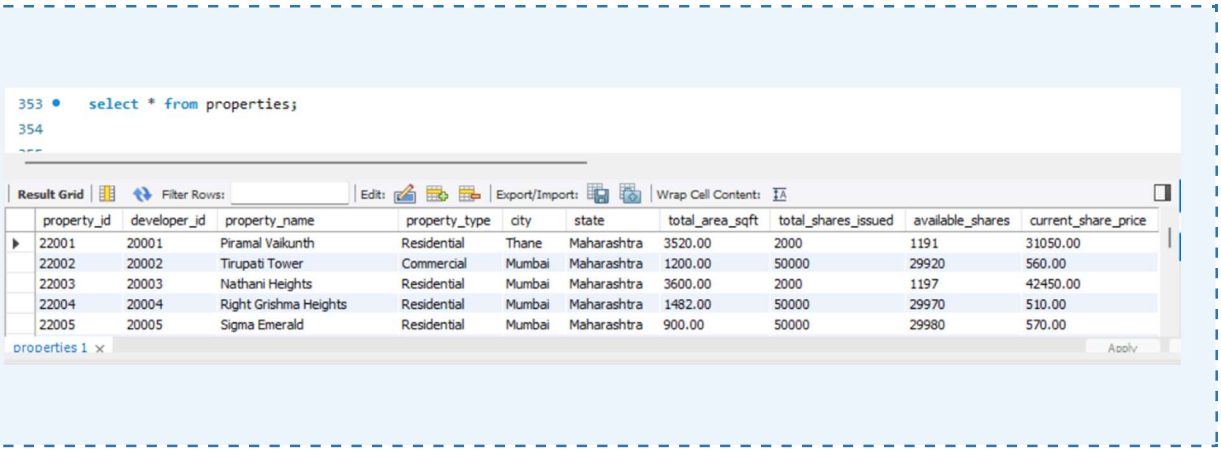
Indexes are created on all foreign-key columns to optimize JOIN performance:

```
CREATE INDEX idx_investors_user        ON Investors           (user_id);
CREATE INDEX idx_developers_user       ON Developers          (user_id);
CREATE INDEX idx_properties_developer  ON Properties          (developer_id);
CREATE INDEX idx_leases_property       ON Lease_Agreements    (property_id);
CREATE INDEX idx_leases_tenant         ON Lease_Agreements    (tenant_id);
CREATE INDEX idx_expenses_property     ON Maintenance_Expenses(property_id);
CREATE INDEX idx_distributions_prop    ON Dividend_Distributions(property_id);
CREATE INDEX idx_ownership_investor    ON Share_Ownership     (investor_id);
CREATE INDEX idx_ownership_property    ON Share_Ownership     (property_id);
CREATE INDEX idx_txns_property         ON Share_Transactions  (property_id);
CREATE INDEX idx_txns_buyer            ON Share_Transactions  (buyer_id);
CREATE INDEX idx_payments_investor     ON Dividend_Payments   (investor_id);
```

9. Database Implementation — DML (Sample Data)

Representative seed data is inserted to demonstrate system behavior. The platform seed includes approximately 200 users (119 investors, 79 developers), 106 properties, 106 tenants, and matching lease and dividend records.

```
-- Admin user
INSERT INTO Users (user_id, full_name, gender, email, password_hash, user_type,
kyc_verified)
VALUES (0, 'System Admin', 'male', 'admin@brickyield.com',
        '$2b$12$...', 'admin', 1);

-- Sample Investor
INSERT INTO Users VALUES (10001, 'Arjun Mehta', 'male', 'arjun@email.com', '...',
'investor', 1);
INSERT INTO Investors VALUES (1, 10001, 250000.00, 0.00, 0.00, 'moderate',
'012345678901');

-- Sample Developer
INSERT INTO Users VALUES (20001, 'Priya Builders Ltd', 'female', 'priya@dev.com',
'...', 'developer', 1);
INSERT INTO Developers VALUES (20001, 20001, 'Priya Builders Ltd', 'REG20001', 0,
1);

-- Sample Property
INSERT INTO Properties (property_id, developer_id, property_name, property_type,
  city, state, total_area_sqft, total_shares_issued, available_shares,
current_share_price)
VALUES (30001, 20001, 'Priya Tech Park', 'Commercial', 'Pune', 'Maharashtra',
        45000.00, 10000, 10000, 250.00);

-- Sample Tenant & Lease
INSERT INTO Tenants VALUES (40001, 'InfoSoft Pvt Ltd', 'Service',
'info@infosoft.com', 4.7);
INSERT INTO Lease_Agreements VALUES (50001, 30001, 40001, 4500.0, 350000.00,
  '2024-01-01', '2026-12-31');

-- Wallet top-up (used in testing)
UPDATE Investors SET wallet_balance = 100000 WHERE risk_profile = 'moderate';
```



```
353 •   select * from properties;
354
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

property_id	developer_id	property_name	property_type	city	state	total_area_sqft	total_shares_issued	available_shares	current_share_price
22001	20001	Piramal Vaikunth	Residential	Thane	Maharashtra	3520.00	2000	1191	31050.00
22002	20002	Tirupati Tower	Commercial	Mumbai	Maharashtra	1200.00	50000	29920	560.00
22003	20003	Nathani Heights	Residential	Mumbai	Maharashtra	3600.00	2000	1197	42450.00
22004	20004	Right Grishma Heights	Residential	Mumbai	Maharashtra	1482.00	50000	29970	510.00
22005	20005	Sigma Emerald	Residential	Mumbai	Maharashtra	900.00	50000	29980	570.00

properties 1 ×

10. SELECT Queries & Views

10.1 Advanced SELECT Queries

Query 1 — Investor Portfolio with Current Market Value

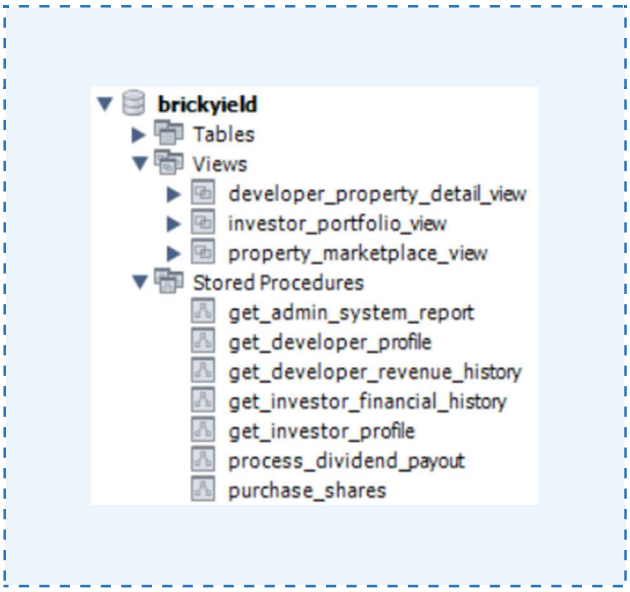
```
SELECT
    i.investor_id, u.full_name, i.wallet_balance,
    SUM(so.shares_owned * p.current_share_price) AS portfolio_value,
    i.total_dividends_earned
FROM Investors i
JOIN Users u          ON i.user_id      = u.user_id
JOIN Share_Ownership so ON so.investor_id = i.investor_id
JOIN Properties p     ON p.property_id  = so.property_id
WHERE so.shares_owned > 0
GROUP BY i.investor_id, u.full_name, i.wallet_balance, i.total_dividends_earned
ORDER BY portfolio_value DESC;
```

Query 2 — Properties with Occupancy Rate & Dividend Yield

```
SELECT
    p.property_id, p.property_name, p.city,
    p.current_share_price,
    ROUND((p.total_shares_issued - p.available_shares) / p.total_shares_issued *
100, 2)
        AS ownership_percent,
    COALESCE(dd.dividend_per_share, 0) AS dividend_per_share,
    ROUND(COALESCE(dd.dividend_per_share,0) / p.current_share_price * 100, 2)
        AS annual_yield_pct
FROM Properties p
LEFT JOIN Dividend_Distributions dd ON p.property_id = dd.property_id
ORDER BY annual_yield_pct DESC;
```

Query 3 — Investors with More Than 3 Properties (HAVING)

```
SELECT
    i.investor_id, u.full_name,
    COUNT(DISTINCT so.property_id) AS properties_held
FROM Investors i
JOIN Users u              ON i.user_id     = u.user_id
JOIN Share_Ownership so   ON so.investor_id = i.investor_id
WHERE so.shares_owned > 0
GROUP BY i.investor_id, u.full_name
HAVING COUNT(DISTINCT so.property_id) > 3
ORDER BY properties_held DESC;
```



10.2 Views

View 1 — investor_portfolio_view

```
CREATE VIEW investor_portfolio_view AS
SELECT
    i.investor_id, u.full_name AS name, u.email, u.kyc_verified,
    i.wallet_balance AS balance, i.total_invested,
    i.total_dividends_earned, i.risk_profile
FROM Investors i
JOIN Users u ON i.user_id = u.user_id;
```

View 2 — property_marketplace_view

```
CREATE VIEW property_marketplace_view AS
SELECT
    p.property_id, p.developer_id, d.company_name AS developer_name,
    p.property_name, p.property_type, p.city, p.state,
    p.total_area_sqft, p.total_shares_issued,
    p.available_shares, p.current_share_price
FROM Properties p
JOIN Developers d ON p.developer_id = d.developer_id;
```

View 3 — developer_property_detail_view

```
CREATE VIEW developer_property_detail_view AS
SELECT p.*, dd.distribution_id, dd.total_revenue, dd.total_expenses,
       dd.net_income, dd.dividend_per_share,
       t.tenant_id, t.tenant_name, t.business_type, t.contact_email AS
tenant_email,
       la.lease_id, la.lease_start_date, la.lease_end_date, la.yearly_rent
FROM Properties p
LEFT JOIN Dividend_Distributions dd ON p.property_id = dd.property_id
LEFT JOIN Lease_Agreements la       ON p.property_id = la.property_id
LEFT JOIN Tenants t                 ON la.tenant_id  = t.tenant_id;
```

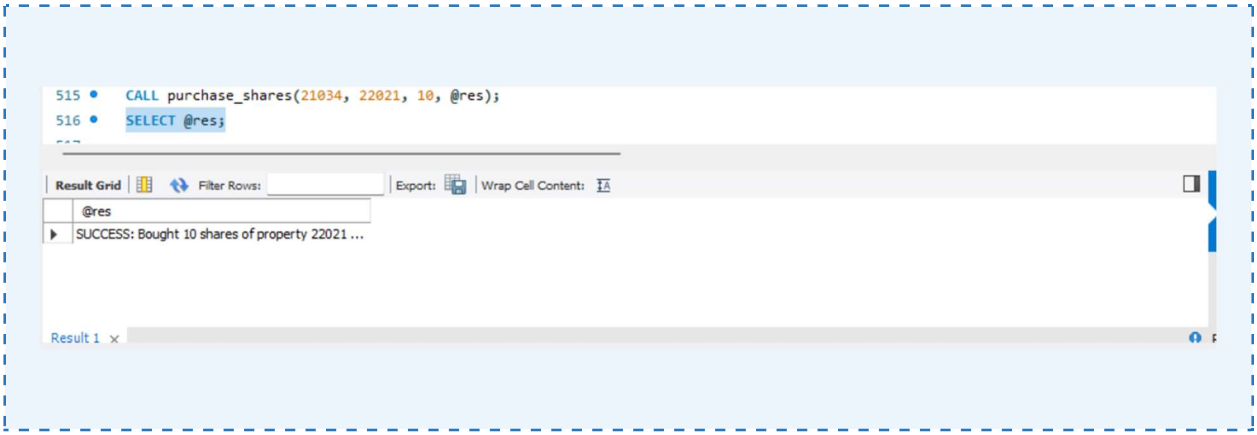
11. Stored Procedures

11.1 purchase_shares — Core Buy Transaction

This procedure is the most critical business transaction on the platform. It is triggered when an investor clicks 'Buy' in the marketplace. It demonstrates START TRANSACTION, SELECT FOR UPDATE (pessimistic locking), ROLLBACK on error, COMMIT on success, and ON DUPLICATE KEY UPDATE for upsert semantics.

```
CREATE PROCEDURE purchase_shares(
    IN  p_investor_id  INT,
    IN  p_property_id  INT,
    IN  p_quantity     INT,
    OUT p_result       VARCHAR(200)
)
BEGIN
    DECLARE v_wallet, v_total_cost   DECIMAL(14,2);
    DECLARE v_share_price            DECIMAL(12,2);
    DECLARE v_available              INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN ROLLBACK; SET p_result = 'ERROR: Transaction rolled back.'; END;

    START TRANSACTION;
    SELECT wallet_balance INTO v_wallet FROM Investors
        WHERE investor_id = p_investor_id FOR UPDATE;
    SELECT current_share_price, available_shares INTO v_share_price, v_available
        FROM Properties WHERE property_id = p_property_id FOR UPDATE;
    SET v_total_cost = p_quantity * v_share_price;

    IF p_quantity <= 0 THEN
        ROLLBACK; SET p_result = 'ERROR: Quantity must be > 0.';
    ELSEIF v_available < p_quantity THEN
        ROLLBACK; SET p_result = 'ERROR: Insufficient shares available.';
    ELSEIF v_wallet < v_total_cost THEN
        ROLLBACK; SET p_result = 'ERROR: Insufficient wallet balance.';
    ELSE
        UPDATE Investors SET wallet_balance = wallet_balance - v_total_cost,
               total_invested = total_invested + v_total_cost
               WHERE investor_id = p_investor_id;
        UPDATE Properties SET available_shares = available_shares - p_quantity
               WHERE property_id = p_property_id;
        INSERT INTO Share_Transactions
            (property_id, buyer_id, transaction_type, shares_quantity,
             price_per_share, total_amount, transaction_date)
        VALUES (p_property_id, p_investor_id, 'buy', p_quantity,
                v_share_price, v_total_cost, CURDATE());
        INSERT INTO Share_Ownership
            (investor_id, property_id, shares_owned, total_invested,
average_purchase_price)
        VALUES (p_investor_id, p_property_id, p_quantity, v_total_cost,
v_share_price)
        ON DUPLICATE KEY UPDATE
            total_invested = total_invested + v_total_cost,
            shares_owned = shares_owned + p_quantity,
            average_purchase_price = (total_invested + v_total_cost)/(shares_owned
+ p_quantity);
        COMMIT;
```

```
        SET p_result = CONCAT('SUCCESS: Bought ', p_quantity, ' shares for Rs.',
v_total_cost);
    END IF;
END;
```



11.2 get_investor_profile

Returns two result sets: (1) the investor's profile card and (2) all property holdings with current market value — used to populate the investor home screen's bubble chart and pie chart.

```
CREATE PROCEDURE get_investor_profile(IN p_investor_id INT)
BEGIN
    -- Result set 1: profile card
    SELECT i.investor_id, u.full_name AS name, u.email,
           i.wallet_balance AS balance, i.total_invested,
           i.total_dividends_earned, i.risk_profile, u.kyc_verified
    FROM Investors i JOIN Users u ON i.user_id = u.user_id
    WHERE i.investor_id = p_investor_id;

    -- Result set 2: holdings for bubbles + pie chart
    SELECT so.ownership_id, so.property_id, p.property_name, p.property_type,
           p.city, so.shares_owned, so.total_invested AS invested_in_property,
           so.average_purchase_price, p.current_share_price,
           so.shares_owned * p.current_share_price AS current_value
    FROM Share_Ownership so JOIN Properties p ON so.property_id = p.property_id
    WHERE so.investor_id = p_investor_id AND so.shares_owned > 0
    ORDER BY so.total_invested DESC;
END;
```

11.3 process_dividend_payout — Cursor-Based Batch Processing

Uses an explicit cursor to iterate over all Dividend_Distributions, insert individual Dividend_Payments, and credit investor wallets in a single atomic batch.

```
CREATE PROCEDURE process_dividend_payout()
```

```
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE cur_dist_id INT; DECLARE cur_prop_id INT;
    DECLARE cur_div_per_share DECIMAL(15,2);
    DECLARE dist_cursor CURSOR FOR
        SELECT distribution_id, property_id, COALESCE(dividend_per_share,0)
        FROM dividend_distributions;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN dist_cursor;
    read_loop: LOOP
        FETCH dist_cursor INTO cur_dist_id, cur_prop_id, cur_div_per_share;
        IF done THEN LEAVE read_loop; END IF;

        INSERT INTO dividend_payments
            (distribution_id, investor_id, shares_held, dividend_amount,
payment_status)
        SELECT cur_dist_id, s.investor_id, s.shares_owned,
               COALESCE(s.shares_owned * cur_div_per_share, 0), 'completed'
        FROM share_ownership s WHERE s.property_id = cur_prop_id;

        UPDATE investors i JOIN share_ownership s ON i.investor_id = s.investor_id
        SET i.wallet_balance = COALESCE(i.wallet_balance,0)
                             + COALESCE(s.shares_owned * cur_div_per_share, 0),
            i.total_dividends_earned = COALESCE(i.total_dividends_earned,0)
                             + COALESCE(s.shares_owned * cur_div_per_share, 0)
        WHERE s.property_id = cur_prop_id;
    END LOOP;
    CLOSE dist_cursor;
END;
```

11.4 get_admin_system_report

Returns two result sets: (1) platform-wide aggregated statistics and (2) per-property revenue/expense data for the admin dashboard chart.

```
CREATE PROCEDURE get_admin_system_report()
BEGIN
    SELECT
        (SELECT COUNT(*) FROM users)                          AS total_users,
        (SELECT COUNT(*) FROM users WHERE user_type='investor') AS total_investors,
        (SELECT COUNT(*) FROM users WHERE user_type='developer') AS
total_developers,
        (SELECT COUNT(*) FROM users WHERE kyc_verified=0)     AS kyc_pending,
        (SELECT COUNT(*) FROM properties)                     AS total_properties,
        (SELECT COALESCE(SUM(total_revenue),0) FROM dividend_distributions) AS
total_revenue,
        (SELECT COUNT(*) FROM lease_agreements
         WHERE CURDATE() BETWEEN lease_start_date AND lease_end_date) AS
active_leases;

    SELECT p.property_name AS name,
           COALESCE(d.total_revenue,0) AS revenue,
           COALESCE(d.total_expenses,0) AS expenses,
           COALESCE(d.net_income,0) AS net
    FROM properties p
    LEFT JOIN dividend_distributions d ON p.property_id = d.property_id;
END;
```

12. Functions & Triggers

12.1 Stored Functions

The following functions encapsulate reusable computation logic:

calculate_investor_roi (p_investor_id INT) → DECIMAL(8,2)

Computes return on investment as (total_dividends_earned / total_invested) × 100. Returns 0.00 if the investor has not yet made any investment.

```
CREATE FUNCTION calculate_investor_roi(p_investor_id INT)
RETURNS DECIMAL(8,2) READS SQL DATA
BEGIN
    DECLARE v_invested  DECIMAL(14,2) DEFAULT 0;
    DECLARE v_dividends DECIMAL(14,2) DEFAULT 0;
    SELECT total_invested, total_dividends_earned
    INTO v_invested, v_dividends
    FROM Investors WHERE investor_id = p_investor_id;
    IF v_invested = 0 OR v_invested IS NULL THEN RETURN 0.00; END IF;
    RETURN ROUND((v_dividends / v_invested) * 100, 2);
END;
```

get_portfolio_value (p_investor_id INT) → DECIMAL(14,2)

Returns current market value of all investor holdings — sum of (shares_owned × current_share_price) across all properties.

```
CREATE FUNCTION get_portfolio_value(p_investor_id INT)
RETURNS DECIMAL(14,2) READS SQL DATA
BEGIN
    DECLARE v_value DECIMAL(14,2) DEFAULT 0.00;
    SELECT COALESCE(SUM(so.shares_owned * p.current_share_price), 0.00)
    INTO v_value
    FROM Share_Ownership so
    JOIN Properties p ON so.property_id = p.property_id
    WHERE so.investor_id = p_investor_id AND so.shares_owned > 0;
    RETURN v_value;
END;
```

calculate_property_yield (p_property_id INT) → DECIMAL(8,2)

Computes annual yield as (net_income / total_property_value) × 100 for a given property.

```
CREATE FUNCTION calculate_property_yield(p_property_id INT)
RETURNS DECIMAL(8,2) READS SQL DATA
BEGIN
    DECLARE v_net_income, v_property_value DECIMAL(14,2) DEFAULT 0;
    SELECT COALESCE(net_income,0) INTO v_net_income
    FROM Dividend_Distributions WHERE property_id = p_property_id LIMIT 1;
    SELECT total_shares_issued * current_share_price INTO v_property_value
    FROM Properties WHERE property_id = p_property_id;
    IF v_property_value = 0 OR v_property_value IS NULL THEN RETURN 0.00; END IF;
    RETURN ROUND((v_net_income / v_property_value) * 100, 2);
END;
```

12.2 Triggers

Triggers are designed to automate audit and validation tasks at the database level without relying on the application layer.

Trigger — Prevent Negative Wallet (BEFORE UPDATE)

```
CREATE TRIGGER before_wallet_update
BEFORE UPDATE ON Investors FOR EACH ROW
BEGIN
    IF NEW.wallet_balance < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR: Wallet balance cannot go negative.';
    END IF;
END;
```

Trigger — Auto-Update available_shares Counter (AFTER INSERT on Share_Transactions)

```
CREATE TRIGGER after_share_buy_update_property
AFTER INSERT ON Share_Transactions FOR EACH ROW
BEGIN
    IF NEW.transaction_type = 'buy' THEN
        UPDATE Properties
        SET available_shares = available_shares - NEW.shares_quantity
        WHERE property_id = NEW.property_id;
    END IF;
END;
```

Trigger — Audit Log on Share_Transactions (AFTER INSERT)

```
-- Conceptual audit trigger (extend schema with a transaction_audit table)
CREATE TRIGGER after_share_transaction_audit
AFTER INSERT ON Share_Transactions FOR EACH ROW
BEGIN
    INSERT INTO transaction_audit (transaction_id, logged_at, action)
    VALUES (NEW.transaction_id, NOW(), 'INSERT');
END;
```

13. Transaction Management & Concurrency Control

13.1 ACID Properties in BrickYield

Property	Guarantee	Implementation in BrickYield
Atomicity	All or nothing	START TRANSACTION + COMMIT/ROLLBACK in purchase_shares, process_dividend_payout
Consistency	Valid state transitions only	FK constraints, CHECK constraints, EXIT HANDLER for SQLEXCEPTION
Isolation	Concurrent transactions do not interfere	SELECT FOR UPDATE (row-level locking) on Investors and Properties rows during purchase
Durability	Committed data survives failures	InnoDB redo log + write-ahead logging; committed rows persist across crashes

13.2 Concurrency Scenarios

Two investors simultaneously trying to purchase shares from the same property is the critical concurrency scenario. BrickYield handles it as follows:

- Transaction A and Transaction B both start, issuing SELECT ... FOR UPDATE on the Properties row.
- MySQL InnoDB grants the row lock to Transaction A first; Transaction B is queued.
- Transaction A deducts shares and commits. Transaction B then acquires the lock and checks available_shares — if insufficient, it rolls back with an informative error message.
- This prevents overselling shares (a critical business invariant) without application-level locking.

13.3 Error Recovery

- All procedures wrap critical DML in a single transaction block with an EXIT HANDLER FOR SQLEXCEPTION.
- On any SQL error (constraint violation, deadlock, etc.), the handler issues ROLLBACK and writes a user-friendly message to the OUT parameter.
- The FastAPI layer captures the OUT parameter value and returns the appropriate HTTP status code (200 for SUCCESS, 400/409 for ERROR messages).

14. Performance Optimization

14.1 Indexing Strategy

Indexes are placed on all foreign-key columns and high-cardinality filter columns. This directly improves JOIN performance for the most frequent queries (portfolio lookup, marketplace listing, developer dashboard).

Index Name	Table	Column(s)
idx_investors_user	Investors	user_id
idx_developers_user	Developers	user_id
idx_properties_developer	Properties	developer_id
idx_leases_property	Lease_Agreements	property_id
idx_leases_tenant	Lease_Agreements	tenant_id
idx_ownership_investor	Share_Ownership	investor_id
idx_ownership_property	Share_Ownership	property_id
idx_txns_buyer	Share_Transactions	buyer_id
idx_payments_investor	Dividend_Payments	investor_id

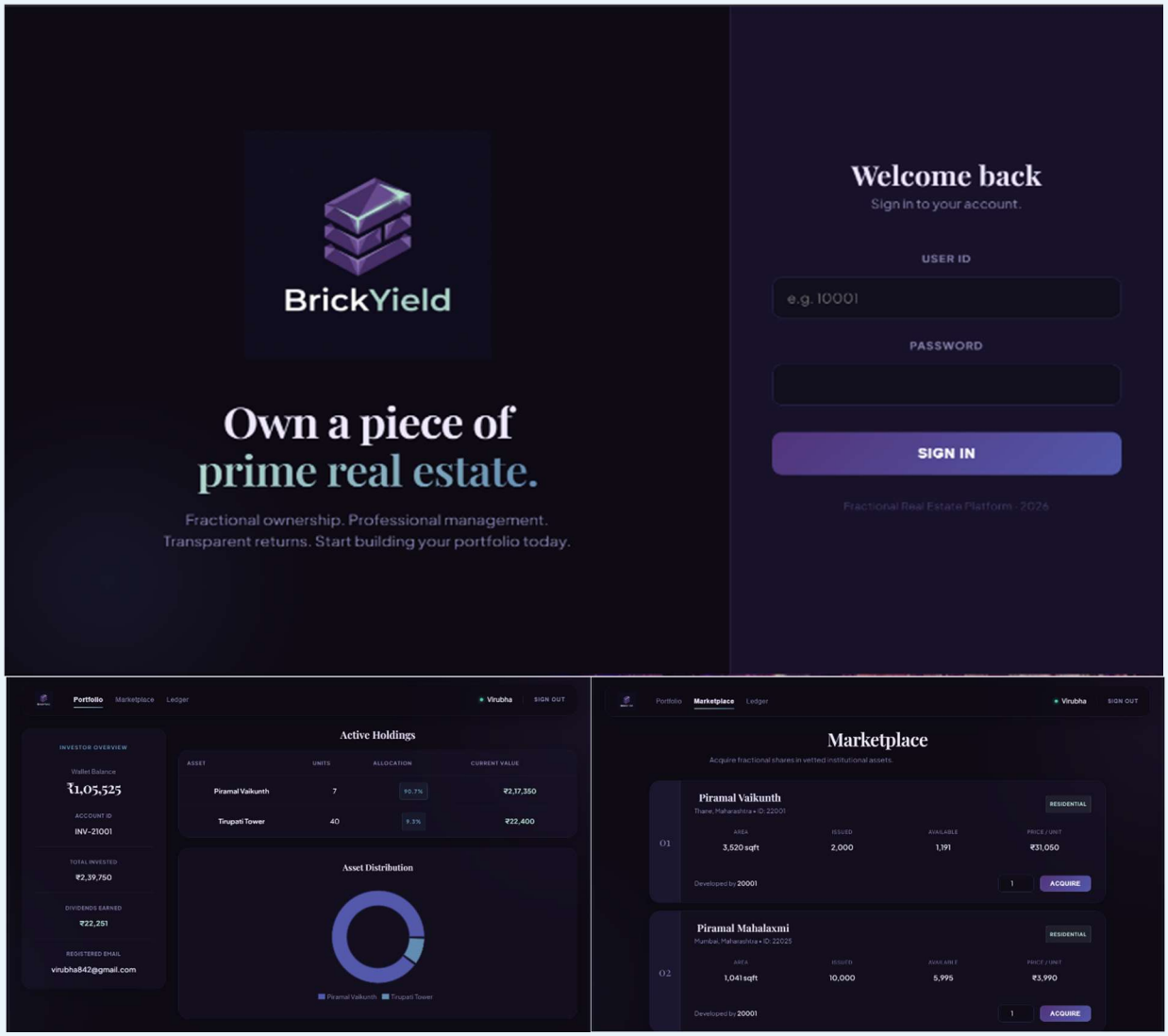
14.2 View-Based Query Optimization

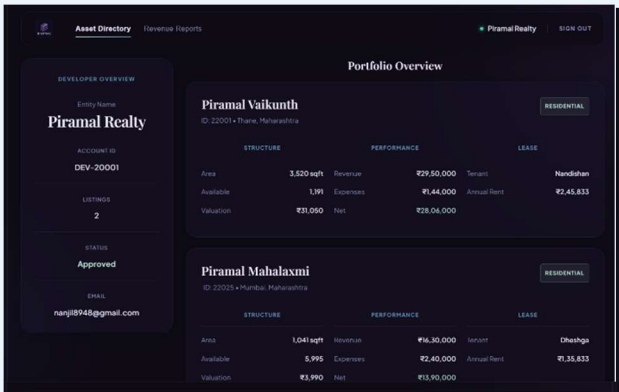
- property_marketplace_view pre-joins Properties and Developers — the API calls one SELECT on the view rather than constructing a JOIN at query time.
- investor_portfolio_view eliminates repeated multi-table JOINs for every investor home screen request.
- developer_property_detail_view aggregates five tables (Properties, Dividend_Distributions, Lease_Agreements, Tenants) into a single view, reducing ORM overhead.

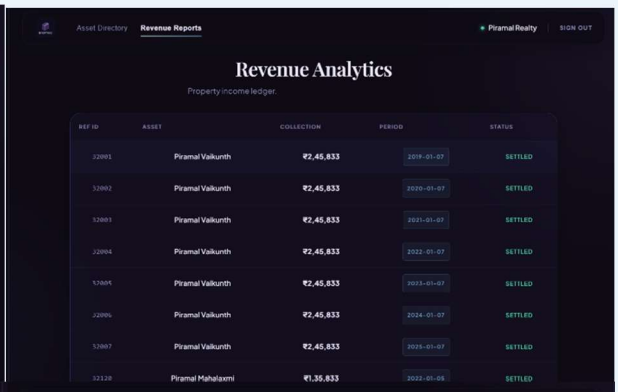
14.3 Schema-Level Optimizations

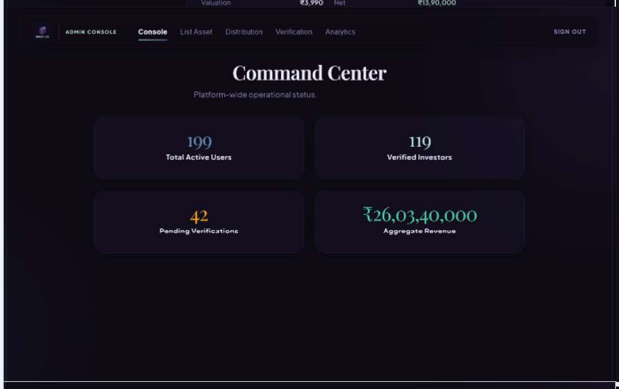
- Denormalization of product_name / price at transaction time in Share_Transactions avoids expensive historical JOINs when generating financial history.
- ENUM types for status fields (transaction_type, payment_status, user_type) store values as integers internally, reducing storage and comparison cost vs. VARCHAR.
- InnoDB's clustered index on primary keys ensures sequential PK reads are I/O efficient.

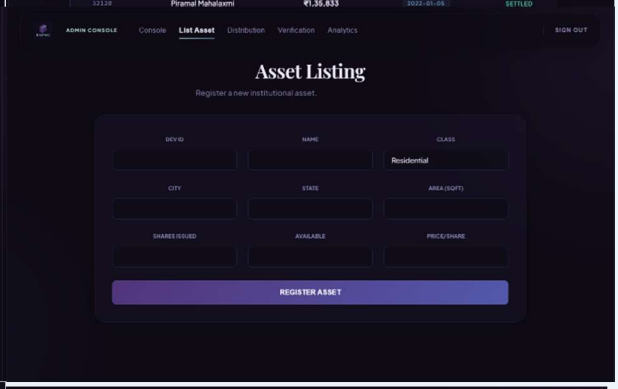
15. Technology Stack

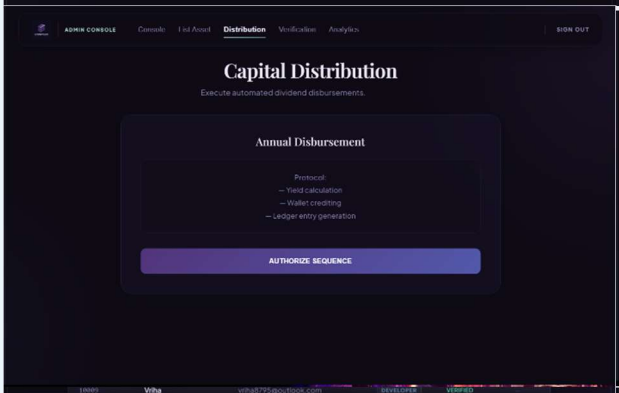


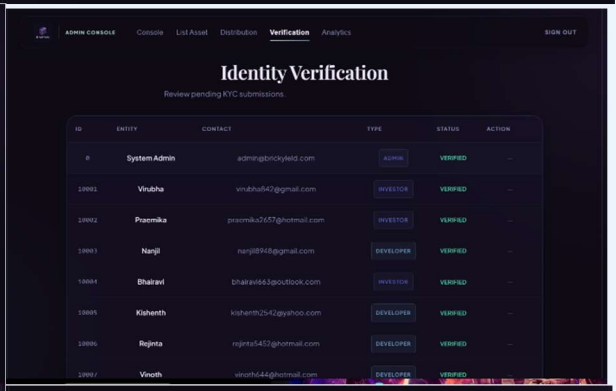

Asset Directory
Revenue Reports
Piramal Realty
SIGN OUT
Portfolio Overview
DEVELOPER OVERVIEW
Entity Name
Piramal Realty
ACCOUNT ID
DEV-20001
LISTINGS
2
STATUS
Approved
EMAIL
nanjil8948@gmail.com
Piramal Vaikunth
ID: 22001 • Thane, Maharashtra
RESIDENTIAL
STRUCTURE
PERFORMANCE
LEASE
Area
3,520 sqft
Revenue
₹29,50,000
Tenant
Nandishan
Available
1,191
Expenses
₹1,44,000
Annual Rent
₹2,45,833
Valuation
₹31,050
Net
₹28,06,000
Piramal Mahalaxmi
ID: 22025 • Mumbai, Maharashtra
RESIDENTIAL
STRUCTURE
PERFORMANCE
LEASE
Area
1,041 sqft
Revenue
₹16,30,000
Tenant
Dheshga
Available
5,995
Expenses
₹2,40,000
Annual Rent
₹1,35,833
Valuation
₹3,990
Net
₹13,90,000

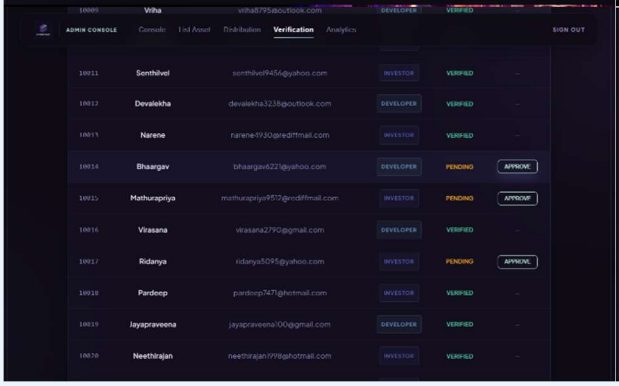

Asset Directory
Revenue Reports
Piramal Realty
SIGN OUT
Revenue Analytics
Property income ledger.
REF ID
ASSET
COLLECTION
PERIOD
STATUS
22001
Piramal Vaikunth
₹2,45,833
2019-01-07
SETTLED
22002
Piramal Vaikunth
₹2,45,833
2020-01-07
SETTLED
22003
Piramal Vaikunth
₹2,45,833
2021-01-07
SETTLED
22004
Piramal Vaikunth
₹2,45,833
2022-01-07
SETTLED
22005
Piramal Vaikunth
₹2,45,833
2023-01-07
SETTLED
22006
Piramal Vaikunth
₹2,45,833
2024-01-07
SETTLED
22007
Piramal Vaikunth
₹2,45,833
2025-01-07
SETTLED
22120
Piramal Mahalaxmi
₹1,35,833
2022-01-05
SETTLED

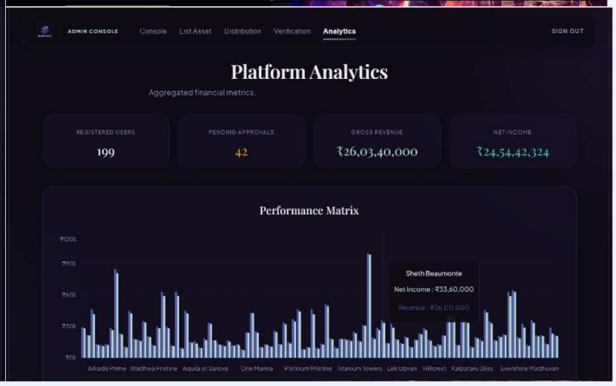

ADMIN CONSOLE
Console
List Asset
Distribution
Verification
Analytics
SIGN OUT
Command Center
Platform-wide operational status
199
Total Active Users
119
Verified Investors
42
Pending Verifications
₹26,03,40,000
Aggregate Revenue


ADMIN CONSOLE
Console
List Asset
Distribution
Verification
Analytics
SIGN OUT
Asset Listing
Register a new institutional asset.
DEV ID
NAME
CLASS
Residential
CITY
STATE
AREA (SQFT)
SHARES ISSUED
AVAILABLE
PRICE/SHARE
REGISTER ASSET


ADMIN CONSOLE
Console
List Asset
Distribution
Verification
Analytics
SIGN OUT
Capital Distribution
Execute automated dividend disbursements.
Annual Disbursement
Protocol:
— Yield calculation
— Wallet crediting
— Ledger entry generation
AUTHORIZE SEQUENCE


ADMIN CONSOLE
Console
List Asset
Distribution
Verification
Analytics
SIGN OUT
Identity Verification
Review pending KYC submissions.
ID
ENTITY
CONTACT
TYPE
STATUS
ACTION
0
System Admin
admin@brickyield.com
ADMIN
VERIFIED
10001
Virubha
virubha642@gmail.com
INVESTOR
VERIFIED
10002
Praemika
praemika2657@hotmail.com
INVESTOR
VERIFIED
10003
Nanjil
nanjil8948@gmail.com
DEVELOPER
VERIFIED
10004
Bhairavi
bhairavi663@outlook.com
INVESTOR
VERIFIED
10005
Kishenth
kishenth2542@yahoo.com
DEVELOPER
VERIFIED
10006
Rejinta
rejinta5452@hotmail.com
DEVELOPER
VERIFIED
10007
Vinoth
vinoth644@hotmail.com
DEVELOPER
VERIFIED


ADMIN CONSOLE
Console
List Asset
Distribution
Verification
Analytics
SIGN OUT
10009
Viha
viha8795@outlook.com
DEVELOPER
VERIFIED
10011
Senthilvel
senthilvel9456@yahoo.com
INVESTOR
VERIFIED
10012
Devalekha
devalekha3238@outlook.com
DEVELOPER
VERIFIED
10013
Narene
narene4930@rediffmail.com
INVESTOR
VERIFIED
10014
Bhaargav
bhaargav6221@yahoo.com
DEVELOPER
PENDING
APPROVE
10015
Mathurapriya
mathurapriya9512@rediffmail.com
INVESTOR
PENDING
APPROVE
10016
Virasana
virasana2790@gmail.com
DEVELOPER
VERIFIED
10017
Ridanya
ridanya5095@yahoo.com
INVESTOR
PENDING
APPROVE
10018
Pardeep
pardeep7471@hotmail.com
INVESTOR
VERIFIED
10019
Jayapraveena
jayapraveena100@gmail.com
DEVELOPER
VERIFIED
10020
Neethirajan
neethirajan1998@hotmail.com
INVESTOR
VERIFIED


ADMIN CONSOLE
Console
List Asset
Distribution
Verification
Analytics
SIGN OUT
Platform Analytics
Aggregated financial metrics.
REGISTERED USERS
199
PENDING APPROVALS
42
GROSS REVENUE
₹26,03,40,000
NET INCOME
₹24,54,42,324
Performance Matrix
Sheth Beaumonte
Net Income : ₹33,60,000
Revenue : ₹36,03,000
Arkade Prime
Wadhwa Pristine
Aquila at Sarova
One Manna
Platinum Pristine
Titanium Towers
Lok Upvan
Hillcrest
Kalpataru Bliss
Evershine Madhuvan

15.1 Frontend — ReactJS

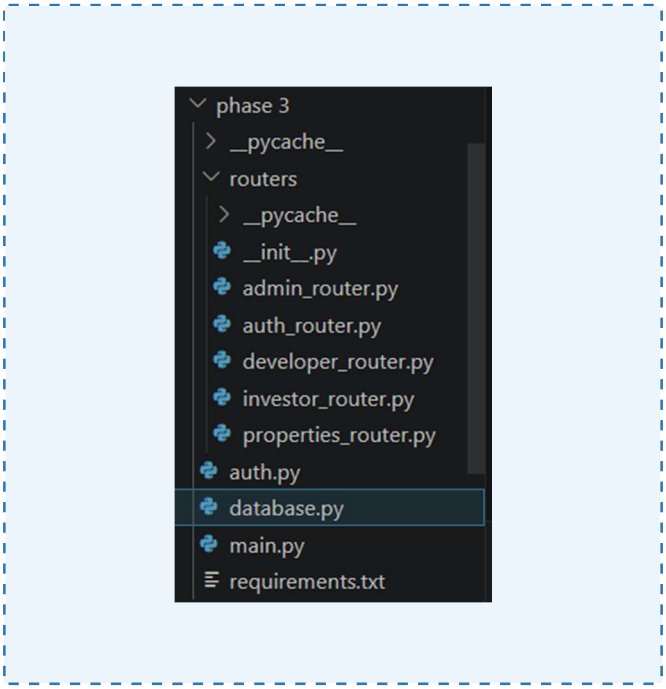
The frontend is built with ReactJS (Vite build tool) and consumes BrickYield's FastAPI endpoints. Key screens include:

- Login Screen — role-based login (User ID + Password), shown in the project screenshot.
- Investor Home — wallet balance, portfolio value, bubble chart of holdings, pie chart allocation.
- Marketplace (Invest Tab) — filterable property grid with developer, city, price, and available-shares data drawn from property_marketplace_view.
- Developer Dashboard — per-property full detail including lease info, revenue, expenses, and dividend distribution.
- Admin Panel — platform statistics and KYC management, asset listing, dividend distribution.

15.2 Backend — FastAPI

FastAPI (Python) acts as the thin middleware between ReactJS and MySQL. It:

- Authenticates requests using JWT tokens with role-based access (investor / developer / admin).
- Calls MySQL stored procedures using the mysql-connector-python driver.
- Parses multi-result-set responses from procedures like get_investor_profile and get_developer_profile.
- Returns structured JSON responses to the frontend.



16. Expected Outcomes & Conclusion

16.1 Expected Outcomes

Upon full deployment, BrickYield will deliver the following outcomes:

19. A normalized, constraint-enforced MySQL database capable of managing thousands of investors, hundreds of properties, and millions of share transactions without data anomalies.
20. Atomic, concurrency-safe share purchase transactions that prevent overselling, double-deduction, or partial updates — enforced by row-level locking and ROLLBACK handlers.
21. Automated dividend distribution through a cursor-driven stored procedure, eliminating manual computation errors.
22. Real-time investor portfolio valuation via efficient indexed JOINs and materialized-style views.
23. A transparent audit trail of every share transaction, dividend payment, and revenue record.
24. A secure, role-based platform connecting local real estate developers with retail investors at minimum share sizes.

16.2 Conclusion

BrickYield demonstrates how a carefully designed relational database can power a sophisticated financial application. By applying normalization principles, enforcing referential integrity through foreign keys, implementing complex business logic in stored procedures, and optimizing reads through views and indexes, the project achieves both correctness and performance.

The combination of MySQL 8.0 (InnoDB), FastAPI, and ReactJS proves that a lean, database-centric architecture can support real-world fractional investment workflows without sacrificing maintainability or scalability. BrickYield shows that practical database management skills — normalization, transaction control, concurrency handling, and performance tuning — are directly applicable to solving real financial inclusion problems.

17. Future Scope

- Secondary Market Trading — enable peer-to-peer share sales with an order-book matching engine implemented as a stored procedure.
- Automated Valuation Model — periodic re-pricing of shares based on rental yield and market comparables.
- Real Payment Gateway Integration — UPI / net banking integration for wallet top-up and dividend withdrawal.
- Property Tokenization — explore blockchain-backed share certificates for immutable ownership records.
- Machine Learning Risk Scoring — replace static risk_profile ENUM with a dynamically computed investor risk score based on portfolio composition and transaction history.
- Mobile App — React Native port of the ReactJS frontend for iOS and Android.
- Regulatory Compliance Module — SEBI / RBI reporting, tax deduction at source (TDS) calculation and Form 16 generation.
- Multi-tenancy Leasing — support for a single property to have multiple tenants (already partially modelled in schema) with pro-rata revenue allocation.

— End of Report —